
Position: The Turing-Completeness of Autoregressive Transformers Relies Heavily on Context Management

Guanyu Cui¹ Zhewei Wei¹ Kun He²

Abstract

Many works make the eye-catching claim that Transformers are Turing-complete. However, the literature often conflates two distinct settings: (i) a *fixed Transformer system* setting, in which a fixed autoregressive Transformer is coupled with a fixed context-management method to process inputs of different lengths step by step, and (ii) a *scaling-family* setting, in which a family of different models (with increasing context-window length or numerical precision) is used to handle different input lengths. Existing proofs of Transformer Turing-completeness are frequently established in setting (ii), whereas real-world LLM deployment and the standard notion of Turing-completeness correspond more naturally to setting (i). In this paper, we first formalize the fixed-system setting, thereby providing a concrete characterization of how real-world LLMs operate. We then argue that results proved in the scaling-family setting provide theoretically meaningful resource bounds but do not establish Turing-completeness, thereby clarifying a common misinterpretation of existing results. Finally, we show that different context-management methods can yield sharply different computational power, and we advocate the position that context management is a central component that critically determines the computational power of real-world autoregressive Transformers.

1. Introduction

The computational power of Transformer variants has been studied extensively, motivated by the striking capabilities

¹Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China ²DEKE Lab, Renmin University of China, Beijing, China. Correspondence to: Zhewei Wei <zhewei@ruc.edu.cn>, Kun He <hekun2023@ruc.edu.cn>.

of Transformer-based large language models (LLMs). A prominent line of work claims that Transformers are Turing-complete (Dehghani et al., 2019; Pérez et al., 2019; Bhatamishra et al., 2020; Giannou et al., 2023; Merrill & Sabharwal, 2024; Li et al., 2024; Roberts, 2024; Back De Luca & Fountoulakis, 2024; Malach, 2024; Nowak et al., 2024; Qiu et al., 2025; Hou et al., 2025; Li & Wang, 2025; Jiang et al., 2026; Schuurmans, 2023; Schuurmans et al., 2024). In other words, Transformers are expressive enough to compute any computable function. However, existing proofs typically rely on assumptions that are not fully justified for real-world LLMs, which correspondingly weakens the solidity of these Turing-completeness claims. This leaves a natural question: *Do the theoretical assumptions made in existing Turing-completeness claims actually apply to autoregressive Transformers as they are used in practice?*

In prior work, it is common to adopt assumptions such as allowing the Transformer context window to grow with the decoding step, or requiring the numerical precision of token embeddings to increase with the number of tokens processed so far. However, such assumptions do not characterize the capability of a single Transformer; rather, they effectively describe the computational power of a collection of different Transformers used at different input lengths. As a result, these works study an object that deviates from the single-Transformer setting we aim to understand, and therefore do not reflect how LLMs are used in practice. Motivated by this observation, we categorize existing works into two regimes depending on whether the Transformer is assumed to be fixed: a *fixed-system* regime and a *scaling-family* regime.

In the fixed-system regime, one fixes a single pretrained Transformer with a fixed context window N , fixed finite numerical precision, and fixed weights, as in real-world Transformer-based LLMs. Given an input token sequence, the model processes it autoregressively by placing tokens into the context window and appending the decoded token to the end of the current sequence until a stopping condition is reached. However, when the length of input or intermediate results exceeds N , the current sequence cannot be fed into the context window directly. In this case, autoregressive decoding must be coupled with a fixed context-management mechanism that determines which tokens are provided to the

context window at each step, thereby enabling the system to process arbitrarily long inputs.

On the other hand, in the scaling-family regime, one considers a family of models with different context-window length, numerical precision, or depth, and uses them to process inputs of different lengths. Many universality results fall into this second regime, even when they are informally interpreted as statements about the first. We summarize several common assumptions that, when adopted, move the analysis away from a single fixed pretrained model and implicitly place it in the scaling-family regime:

- **Scaling or unbounded window:** assuming that inputs of arbitrary length fit into the context window, or that during autoregressive decoding each token can attend to the entire input and all previously generated tokens.
- **Scaling or unbounded precision:** assuming internal representations require precision that grows with the input length, for example log-precision, or directly using unbounded rationals or real numbers.

In contrast to the above assumptions, a real-world Transformer has a fixed maximum context-window length and fixed internal numerical precision that does not vary with the input length. Hence, once these assumptions are adopted, the object of study is no longer a fixed real-world Transformer, but rather a family of Transformers, where different Transformers are used for sequences of different lengths.

Position and Contributions. We argue that theoretical studies of the computational power of Transformers should explicitly state their assumptions and distinguish scaling-family settings from realistic fixed-system settings. Scaling or unbounded context windows, as well as growing or unbounded numerical precision, change the object of study from a single deployed Transformer system to a family of models. For real-world LLMs, **the way context management, or more broadly the model’s “harness”, is coupled with the Transformer is not a peripheral implementation detail but a central component that can fundamentally change the induced computational model.**

We support this position through three contributions:

- We formalize a computational model for a fixed Transformer system. This provides a reference setting for discussing real-world autoregressive LLM deployment.
- We distinguish fixed-system and scaling-family regimes and explain how common assumptions such as scaling context windows and growing numerical precision should be interpreted. Results in the scaling-family regime are meaningful, but they do not establish Turing-completeness of a fixed deployed Transformer system.
- Under the fixed-system formalization, we provide simple derivations showing that summarization-style context management yields only constant-space computation, while appending-style methods achieve linear space; more sophisticated mechanisms can even be Turing-complete. These examples illustrate why context management is a central component of the induced computational model.

Related Work. Our work is primarily motivated by a blog post (Akhlaghpour, 2024) that informally argues that many Turing-completeness claims for Transformers involve implicit issues. However, the blog post is informal and presents these concerns by surveying related works in chronological order. In contrast, we provide an explicit formalization, clearly separate existing settings into the fixed-system and scaling-family regimes, and highlight a key observation largely absent from the blog discussion: how the choice of context manager can substantially affect the computational power of real-world LLM systems.

Roadmap. Section 2 reviews the background needed for our formalization and analysis. Section 3 defines the computational process of a fixed Transformer system (T, D, C) . Section 4 explains why interpreting scaling-family results as “autoregressive Transformers are Turing-complete” reflects a misconception, and surveys representative claims by identifying the scaling assumptions they rely on. Section 5 analyzes the computational power and separations of fixed Transformer systems under different context-management methods. Finally, Section 6 summarizes our conclusion and presents a call to action.

2. Preliminaries

This section reviews the background concepts that will be used later: (i) Turing machines (TMs), transducers, and complexity classes, (ii) Boolean circuits, and (iii) a minimal abstraction of Transformers and autoregressive decoding.

2.1. Notation

Let Σ be a finite alphabet. A string over Σ is an element of $\Sigma^* := \bigcup_{\ell=0}^{\infty} \Sigma^\ell$, where $\Sigma^0 = \{\epsilon\}$ and ϵ denotes the empty string. For a string x , let $|x|$ denote its length. For integers $1 \leq a \leq b \leq |x|$, we write $x_{a:b}$ for the substring $x_a \cdots x_b$. A language L over Σ is a set of strings, i.e., $L \subseteq \Sigma^*$. A partial function from A to B , denoted $g : A \rightharpoonup B$, is a mapping that may be undefined for some $a \in A$.

2.2. TMs, Transducers, and Complexity Classes

Turing machines (TMs) are a classical computational model proposed by Alan Turing (Turing, 1937) to formalize the notion of computability. Informally, a (multi-tape) Turing machine consists of k tapes, each extending infinitely

in one direction and divided into cells that store symbols. Each tape is equipped with a read-write head that can move left or right. Initially, the input is written on the tape(s) according to a fixed convention, and all other cells contain a blank symbol. At each step, the machine reads symbols, updates its state, writes symbols, and moves the heads according to a finite set of rules. A **transducer** is a machine that maps an input string to an output string; one convenient model is a TM with a read-only input tape, a work tape, and a write-only output tape. We use standard complexity classes such as $\text{DTIME}(t(n))$ and $\text{DSPACE}(s(n))$ for languages that can be decided deterministically in $O(t(n))$ time or $O(s(n))$ auxiliary space. We use $\text{FDTIME}(t(n))$ and $\text{FDSPACE}(s(n))$ for the corresponding function problems. When discussing space complexity, we can ignore the constant number of tapes, since any k -tape Turing machine can be simulated by a single-tape one using at most a factor- k increase in space.

2.3. Boolean Circuits

A **Boolean circuit** is a computational model for a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$. It can be represented as a finite directed acyclic graph (DAG) with n source nodes $x_1, \dots, x_n$ as inputs and m sink nodes $y_1, \dots, y_m$ as outputs. All other nodes are logic gates such as AND, OR, NOT, and edges are wires carrying bits. Given an input assignment $x \in \{0, 1\}^n$ to the input nodes, each gate is evaluated by applying its operation to the values on its incoming wires, and the values at the output nodes define $f(x) \in \{0, 1\}^m$. Unlike a single Turing machine which can process inputs of arbitrary length, a single Boolean circuit only handles inputs of a fixed length. Therefore, in circuit complexity, we typically study a **family** of circuits $\{C_n : n \in \mathbb{N}\}$, where C_n handles inputs of length n .

2.4. Turing-Completeness

Turing-completeness is a notion used to characterize whether a class of objects is at least as expressive as Turing machines. To define Turing-completeness, we first define Turing-computable functions. A partial function $f : \Sigma^* \rightarrow \Sigma^*$ is **Turing-computable** if there exists a TM M such that for all $x, y \in \Sigma^*$, $f(x) = y$ if and only if M halts on input x and outputs y . A collection of objects $\mathcal{N}$ is **Turing-complete** if for every Turing-computable partial function f there exists an object $N \in \mathcal{N}$ that computes f . From this definition, a necessary condition for Turing-completeness is that each object supports inputs of unbounded length.

2.5. Transformers

The Transformer (Vaswani et al., 2017) model is a well-known neural network architecture designed for sequence

modeling which relies primarily on self-attention mechanism to capture the content of the sentence and to generate results. Transformer models can be broadly categorized by whether they include encoder layers and/or decoder layers into three families: encoder-only Transformers (e.g., BERT (Devlin et al., 2019), RoBERTa (Liu et al., 2019)), encoder-decoder Transformers (e.g., the vanilla Transformer (Vaswani et al., 2017), T5 (Raffel et al., 2020), BART (Lewis et al., 2020a)), and decoder-only Transformers (e.g., GPT (Brown et al., 2020; Achiam et al., 2023), LLaMA (Touvron et al., 2023), Gemini (Gemini Team et al., 2023; 2024), DeepSeek (DeepSeek-AI et al., 2024; Guo et al., 2025), and Qwen (Bai et al., 2023; Yang et al., 2025)).

Given an input sentence, a tokenizer is applied before the Transformer to map the raw text to a sequence of discrete tokens from a finite vocabulary Σ . This token sequence is then fed into the Transformer. In practice, tokens are mapped to token IDs before being input to the Transformer. Since this is only an encoding choice, we still treat Σ as the token domain. A pretrained decoder-only Transformer with context window length N and fixed numerical precision can be abstracted as a fixed function T that maps an N -token sequence¹ to next-token logits, and hence to a distribution over Σ . Formally, $T : \Sigma^N \rightarrow \Delta(\Sigma)$, where $\Delta(\Sigma)$ denotes the set of all distributions on Σ . As a supplement, many Transformer architectures use positional encodings to incorporate token-position information, either at the embedding stage (e.g., the sinusoidal PE in the vanilla Transformer (Vaswani et al., 2017) or the learned PE in BERT (Devlin et al., 2019)) or within self-attention (e.g., RoPE (Su et al., 2024)). In this case, we may regard the input within the context window as a length- N string over $\Sigma \times \mathcal{P}$, where $\mathcal{P} = \{0, 1\}^{p_{\text{pos}}}$ for a fixed p_{pos} . Thus, using positional encodings only enlarges the alphabet by a constant factor, and we will not mention them separately in what follows. For our purposes, the architectural details of attention are not essential and are deferred to Appendix A. What matters is that (i) the input length (the context-window length) of a single Transformer is a fixed constant N , and (ii) all parameters and internal representations use fixed finite precision.

2.6. Autoregressive Decoding and Context Management

Decoder-only Transformers are typically used via autoregressive decoding. Given a current token sequence $x = x_1 \dots x_\ell$, we feed x into the Transformer T to obtain a next-token distribution $T(x)$, apply a decoding rule D to select the next token $x_{\ell+1}$, and append it to the end of x to form the new sequence, iterating this procedure. This process is

¹Although the Transformer’s input length is fixed to N , in practice a single forward pass can process any input sequence of length at most N . If the input is shorter than N , it is padded with $\langle \text{pad} \rangle$. When we say that a sequence of length $< N$ is placed into the context window, we omit the padding step for brevity.

well defined when the sequence length does not exceed the context-window length N . To continue decoding when the sequence length exceeds N , one must introduce a context-management mechanism C that determines which tokens are provided to the context window at each step. Section 3 formalizes this computation process as a fixed system.

3. The Fixed-System Regime for Transformers

We now formalize computation of autoregressive Transformers in the fixed-system regime. The goal is to make explicit what is fixed and what can grow with the decoding steps.

3.1. The System (T, D, C) and Its Execution

Fix a finite token vocabulary Σ and a context window length N . A *fixed Transformer system* is a triple (T, D, C) :

- T is a fixed pretrained Transformer, viewed as a function that maps any string $w \in \Sigma^N$ in its context window to a distribution $\mu_w \in \Delta(\Sigma)$ over the next token.
- D is a deterministic² constant-time decoding rule that maps a distribution over Σ to an output token in Σ . Typical choices include greedy decoding ($\arg \max$) or a fixed-temperature softmax sampling with a fixed seed.
- C is a deterministic context manager. At step t , it maintains internal state and strings $r^{(t)} \in \Sigma^*$, produces a context string $w^{(t)} \in \Sigma^N$ via $w^{(t)} = C_w(r^{(t)})$, and then updates its state and strings after decoding $\hat{x}_{t+1}$ by setting $r^{(t+1)} = C_r(\hat{x}_{t+1}, r^{(t)}) \in \Sigma^*$.

Given an input string $x = x_1 x_2 \cdots x_n \in \Sigma^*$, we feed it to C and set $r^{(1)} = x$. The system then repeatedly applies $w^{(t)} = C_w(r^{(t)})$ to form the token sequence fed into the context window, $\hat{x}_{t+1} = D(T(w^{(t)}))$ to obtain the next token, and $r^{(t+1)} = C_r(\hat{x}_{t+1}, r^{(t)})$ to update its maintained strings, until a stopping condition of C is met. The system output is the generated token sequence (or a final answer extracted by a fixed convention). For decision problems, we can designate special accept and reject tokens and say that the system decides a language by eventually emitting exactly one of them. For function computation, we view the system as a transducer that outputs a string.

3.2. Some Common Context-Management Methods

We summarize several common context-management methods that will be referenced later.

²For simplicity, we assume the next-token selection rule is deterministic and single-valued. If not, one can adopt a nondeterministic-TM-style convention and define the Transformer’s output as the set of all valid next tokens.

Summarization-Style. A common approach is to compress earlier parts of the history via summarization, e.g., latent compression methods such as AutoCompressor (Chevalier et al., 2023) and ICAE (Ge et al., 2024), or the `/compress` or `/compact` commands as used in Gemini-CLI (Google, 2026a), OpenAI Codex CLI (OpenAI, 2026a), and Claude Code (Anthropic, 2026a), which replaces part of the earlier context with a shorter summary. Given an input sequence $x = x_1 \cdots x_n \in \Sigma^*$ and setting $r^{(1)} = x$, an example summarization-style context-management process operates as follows:

- **Normal decoding phase.** If $|r^{(t)}| < N - 1$, the context manager provides $w^{(t)} = r^{(t)}$ to the Transformer. Given the decoded next token $\hat{x}_{t+1}$, there are two cases. If $\hat{x}_{t+1}$ is the end-of-sequence token $\langle \text{EOS} \rangle$, the system terminates. Otherwise, set $r^{(t+1)} = r^{(t)} \circ \hat{x}_{t+1}$.
- **Summarization phase.** If $|r^{(t)}| \geq N - 1$, the context manager provides $w^{(t)} = \langle s \rangle \circ r_{1:N-1}^{(t)}$ to the Transformer, where $\langle s \rangle$ is a summary-instruction token that triggers summarization. The Transformer then decodes $\hat{x}_{t+1}$ as a summary of the contents in the window, and we update $r^{(t+1)} = \hat{x}_{t+1} \circ r_{N-|r^{(t)}|}^{(t)}$. For simplicity, we assume that the summary is a single token. To generate a multi-token summary, we can reserve a budget of t tokens (with $1 < t \leq N/2$) in the context window, that is, trigger summarization when $|r^{(t)}| \geq N - t$ instead of $N - 1$. Once summarization is triggered, the model can generate a length- $< t$ summary autoregressively (i.e., by invoking the normal decoding phase as a subroutine) and prepend the resulting summary sequence to $r^{(t)}$.

Figure 1 illustrates the summarization-style context manager described above.

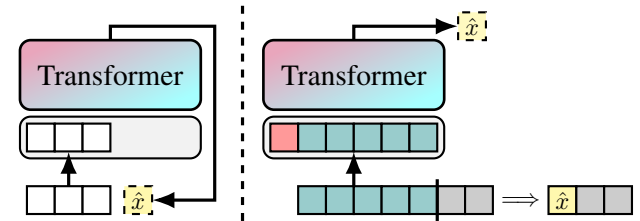


Figure 1. Illustration of summarization-style context management. Left: normal decoding phase. Right: summarization phase. Teal tokens indicate the segment to be summarized, the red token is the summary-instruction token $\langle s \rangle$, and the yellow token denotes the generated summary $\hat{x}$.

Appending-Style. Another method, similar to that used by Schuurmans et al. (Schuurmans et al., 2024), treats the Transformer’s context window as a sliding window and appends predicted tokens to the end of the sequence. Given an input sequence $x = x_1 \cdots x_n \in \Sigma^*$ and setting $r^{(1)} = x$, an example appending-style context-management process

operates as follows. At step t , the context manager provides $w^{(t)} = r_{1:\min\{|r^{(t)}|, N\}}^{(t)}$ to the Transformer and obtains $\hat{x}_{t+1}$. If $\hat{x}_{t+1}$ is a halting token in a designated set H , the system terminates. Otherwise, it appends the decoded token to the end of $r^{(t)}$ and shifts the window forward by one token, i.e., $r^{(t+1)} = r_{2:|r^{(t)}|}^{(t)} \circ \hat{x}_{t+1}$. Figure 2 illustrates the appending-style context manager described above.

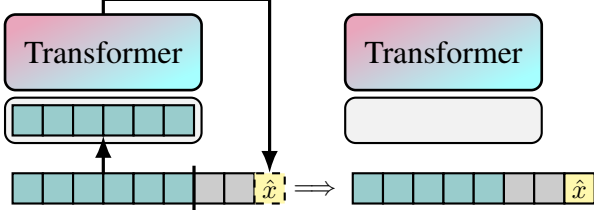


Figure 2. Illustration of appending-style context management. Left: decode a new token $\hat{x}$ to append to the end of the sequence. Right: shift the window forward by 1 token.

Other Methods and Remark on the Power of C . Beyond the two methods mentioned above, many other context-management mechanisms have been proposed. For example, many systems write important information to external storage and retrieve it when needed, as in retrieval-augmented generation and external-memory-based agents (Guu et al., 2020; Lewis et al., 2020b; Packer et al., 2023). Other systems, such as ToolFormer (Schick et al., 2023), GPT (OpenAI, 2026b), Claude (Anthropic, 2026b), and Gemini (Google, 2026b), also incorporate tool calls or function calls to access external resources. We remark that the definition of a context manager C is very broad. With such an unrestricted C , the overall system can become trivially powerful. For example, if C can call a Turing-complete tool (e.g., by executing Python programs), then (T, D, C) is Turing-complete regardless of T . Therefore, when we analyze several of these methods in Section 5, we explicitly restrict the capabilities of C , while still covering common deployed mechanisms such as summarization and sliding windows.

4. Alternative Views

Many works make (or are commonly interpreted as making) the claim: *Autoregressive Transformers are Turing-complete*. To interpret this claim precisely, one must specify the computational regime. In the fixed-system regime, Turing-completeness would mean that for any Turing machine, there exists a single fixed system (T, D, C) can correctly simulate it on inputs of arbitrary length. In contrast, in the scaling-family regime, it suffices to prove that for each input length n (or number of steps t), there exists a Transformer model whose context-window length or numerical precision depends on n (or t) that simulates the desired computation. Only the fixed-system regime matches the way

real-world LLMs are deployed, where the pretrained model is fixed and inputs are handled by a fixed context-manager.

In this section, we first clarify why Turing machine simulation results proved in the scaling-family regime should not be interpreted as establishing Turing-completeness. We then survey representative works and identify whether they implicitly adopt the assumptions listed in Section 1, which place their analysis in the scaling-family regime and therefore do not by themselves establish Turing-completeness of a fixed autoregressive Transformer system.

4.1. Scaling Models Does Not Mean Turing-Complete

By Definition 2.4, to establish Turing-completeness for autoregressive Transformers in the fixed-system regime, one must show that for every Turing-computable partial function f there exists a single system (T, D, C) that computes f on inputs of unbounded length. In contrast, showing that longer inputs or longer decoding runs can be simulated by a scaled model only yields a resource bound for Transformers to handle different input lengths. This is closer in spirit to a circuit-complexity result. Savage (Savage, 1972) showed that if a language L is decidable in time $\text{DTIME}(T(n))$, then for each n there exists a circuit C_n of size $O((T(n))^2)$ that decides L on length- n inputs. Scaling circuit size with n does not make a single circuit universal. It produces a family of circuits, one for each input length. Similarly, scaling the context-window length or the numerical precision yields a scaling-family viewpoint rather than Turing-completeness of a fixed pretrained Transformer system. Beyond this conceptual misalignment, real-world LLMs operate with a fixed finite context window and rely on context management to handle inputs of arbitrary length, whereas the scaling-family regime assumes that arbitrarily long histories can be placed into the context window. This also departs from how LLMs are used in practice.

4.2. Works That Scale Transformers

In this subsection, we review representative works that claim Turing-completeness for Transformers, or claim that Turing machines can be simulated by Transformers, and identify which ones explicitly or implicitly fall into the scaling-family regime. We group these works into two categories.

Group A: Scaling Window. A large fraction of “Transformer simulates TM” arguments implicitly assume that, given an input sequence of length n , at decoding step t the token at position $n + t$ can attend to all tokens in the sequence within a single Transformer forward pass. This effectively requires the context-window length at step t to be at least $n + t$. Works that explicitly or implicitly adopt this assumption include (Dehghani et al., 2019; Pérez et al., 2019; Bhattamishra et al., 2020; Merrill & Sabharwal, 2024;

Li et al., 2024; Roberts, 2024; Malach, 2024; Nowak et al., 2024; Qiu et al., 2025; Hou et al., 2025; Jiang et al., 2026). This group also includes looped or recurrent Transformer variants that repeatedly modify a fixed length input. Although the context-window length may not grow with the number of decoding steps in these formulations, handling longer inputs still requires a model with a larger context window. Thus, these works are also scaling the window in essence. Works that explicitly or implicitly adopt this assumption include (Giannou et al., 2023; Back De Luca & Fountoulakis, 2024; Li & Wang, 2025).

Group B: Scaling Precision. Several constructions require the precision of token embeddings or internal representation matrices to grow with the input length n or the number of decoding steps t , or they assume exact unbounded-precision real or rational arithmetic. This is incompatible with the fixed finite precision of deployed models. Works that explicitly or implicitly adopt this assumption include (Dehghani et al., 2019; Pérez et al., 2019; Bhattamishra et al., 2020; Giannou et al., 2023; Merrill & Sabharwal, 2024; Li et al., 2024; Roberts, 2024; Back De Luca & Fountoulakis, 2024; Nowak et al., 2024; Qiu et al., 2025; Hou et al., 2025; Jiang et al., 2026).

Table 1 summarizes the assumptions used in related works. Regarding the context-window length, all related works in the scaling-family setting assume a non-constant context window. Regarding numerical precision, only two works assume constant precision. These misalignments make the corresponding Turing-completeness claims more subtle and arguably debatable.

4.3. Works with a Fixed Transformer

Two works highlight that, even with fixed numerical precision and a fixed context window, computational universality can arise by modifying the overall system, especially the context-management interface.

1. Schuurmans (Schuurmans, 2023) claims that a fixed Transformer-based LLM will be Turing-complete when augmented with access to external memory. The paper provides experimental evidence that a real LLM can simulate all single-step operations of a Turing-complete language, but it does not give a constructive theoretical proof of how a Transformer realizes these operations.
2. Schuurmans et al. (Schuurmans et al., 2024) propose an extended autoregressive decoding system with a fixed context window that can be Turing-complete by generating up to two output tokens per step and appending them to the token sequence for future processing.

These works suggest that, in the fixed Transformer system

Table 1. Assumptions used in representative works. “Window” denotes the context-window length, i.e., the sequence length processed by a single self-attention operation. “Precision” denotes the number of bits used to represent one token. Here n denotes the initial input length, t denotes the number of decoding steps (often interpreted as the CoT length), and $s(n)$ denotes the space complexity to recognize the language. “Unbounded” precision means the proof assumes exact real / rational computation without a concrete precision bound.

Window	Precision	Work
$n + t$	unbounded	(Dehghani et al., 2019)*, (Pérez et al., 2019; Bhattamishra et al., 2020; Roberts, 2024; Nowak et al., 2024; Jiang et al., 2026)
	$\text{poly}(n)$	(Li et al., 2024) [†]
	$O(\log(n + t))$	(Merrill & Sabharwal, 2024; Li et al., 2024) [†] , (Qiu et al., 2025), (Hou et al., 2025)*
	$O(1)$	(Malach, 2024) ^{*†}
n	unbounded	(Back De Luca & Fountoulakis, 2024)
	$O(\log n)$	(Giannou et al., 2023)
$s(n)$	$O(1)$	(Li & Wang, 2025)

* The original paper does not explicitly state an upper bound on the numerical precision, to the best of our understanding.

† The original paper does not explicitly claim that Turing-completeness has been proved; instead, it claims that one can construct a Transformer that simulates the execution of a Turing machine on a given input instance.

regime, Turing-completeness may be achieved by modifying the decoding procedure or designing different context-management mechanisms, so scaling the Transformer itself is not necessary. Section 5 further analyzes how different context-management methods affect the computational power of a fixed Transformer system.

5. Our Position: Context Management Matters for Autoregressive Transformers

In Section 4, we point out that existing studies often conflate settings that align with real-world Transformers with settings that depart from them, while broadly claiming that Transformers are Turing-complete. Our position is different. We argue that, in the fixed Transformer system setting that better reflects real-world deployed Transformers, the method of context management has a greater impact on the system’s computational power than the Transformer itself.

To support our view, we use simple theoretical derivations to analyze how different context-management methods affect the upper bound of the computational power of the entire system. Specifically, we examine two context-management methods, namely summarization-style context management and appending-style context management. We prove that, when combined with a fixed-context-window, fixed-precision Transformer, summarization-style context management leads to a system bounded by a constant-space Turing machine, whereas appending-style context management yields a system with the same power as a linear-space Turing machine. In addition, drawing on existing works, we point out that the resulting systems are Turing-complete if the Transformer is allowed to read from and write to memory through a context-management mechanism, or if the Transformer is allowed to decode one or two tokens at a time.

Through the survey in Section 4.2 and Section 4.3, we show that most papers focus only on the importance of the Transformer itself while overlooking the importance of harnesses such as context management. Therefore, our position paper highlights an issue that has been overlooked by the community and provides corrective guidance for subsequent research in this direction by emphasizing the importance of context management.

5.1. Analysis of Different Context-Management Methods

Next, we discuss how different context-management methods affect the computational power of Transformer systems with a fixed context-window size and fixed precision. As discussed in Section 3, an unrestricted context manager C can make the overall system trivially powerful. Accordingly, we focus on “simple enough” context managers. Such a manager maintains only N memory cells for the Transformer’s context window and $O(1)$ auxiliary memory cells for states, beyond storing token strings organized in simple data structures such as queues. It can apply only fixed local operations to these maintained strings, for example extracting a prefix by popping a constant number of tokens, shifting a window by a constant amount, appending tokens by pushing them to the tail, and querying the Transformer for the next token. This convention captures the summarization-style and appending-style context managers described in Section 3.2, while excluding the ability to run general algorithms.

Summarization-Style Context Management. Next, we show that the system induced by summarization-style context management is upper bounded by a deterministic constant-space Turing machine, or $\text{FSPACE}(1)$ ³.

³Strictly speaking, space complexity classes are defined for machines that halt on all inputs. A machine using bounded space need not halt, so our use of space-complexity notation is a mild

Proposition 5.1. *Any fixed Transformer system (T, D, C) , where C is a summarization-style context manager, can be simulated by a Turing machine using constant space.*

Proof. Assume T has context-window length $N = O(1)$ tokens, with each input token in Σ . An observation that will be used implicitly in what follows is that a single decoding step of a Transformer, $D(T(\cdot)) : \Sigma^N \rightarrow \Sigma$, can be simulated by a Turing machine in constant space (with the constant depending on N).

We construct a one-way transducer to simulate each step t of the computation process of the system (T, D, C) , where the head of the read-only input tape never moves left (recall that a transducer can be formalized as a three-tape Turing machine with a read-only input tape, a read/write work tape, and a write-only output tape).

Initialization. At initialization, the input tape contains the input string $x = x_1 \cdots x_n \in \Sigma^*$, the other two tapes are blank, and all heads start at the first cell. We write a boundary marker $\#$ in cell $N + 1$ of the work tape to partition the tape into two parts: cells 1 through N simulate the contents of the Transformer’s context window $w^{(t)}$, while cell $N + 2$ onward is the workspace region used to simulate a single Transformer decoding step $D(T(w^{(t)}))$.

Simulating step t of the system. At the beginning of the simulation of the t -th step of a Transformer system with summarization-style context management, we copy (token by token) the symbol under the input head and append it to the end of the token string on the work tape, until either the number of tokens on the work tape reaches $N - 1$ or the input tape is exhausted. We then distinguish two cases based on the number of tokens currently on the work tape:

- If the number of tokens on the work tape is less than $N - 1$ (denote the length by $\ell < N - 1$), we simulate one Transformer decoding step $D(T(\cdot))$ in the workspace region using these tokens as input. We then write the decoded token $\hat{x}_{t+1}$ into cell $\ell + 1$, and finally clear the workspace region.
- If the number of tokens on the work tape is $N - 1$, we shift these tokens one cell to the right and write the summary token $\langle s \rangle$ in the first cell. We then simulate one Transformer decoding step $D(T(\cdot))$ in the workspace using these length- N token sequence, write the decoded token $\hat{x}_{t+1}$ into the first cell of the work tape, and finally clear the work tape so that only $\hat{x}_{t+1}$ in the first cell and the boundary marker $\#$ in cell $N + 1$ remain.

The transducer then proceeds to simulate step $t + 1$ of the system. It is easy to see that, during the simulation of any

abuse adopted for notational convenience. The same caveat applies throughout.

step t , the token sequence on the work tape (excluding the boundary marker $\#$), concatenated with the token sequence on the input tape at and to the right of the input head, is always equal to $r^{(t)}$. The first case, in which placing all of $r^{(t)}$ into the context window yields a length no greater than $N - 1$, corresponds exactly to $|r^{(t)}| < N - 1$, while the second case corresponds to $|r^{(t)}| \geq N - 1$. In both cases, the Turing machine simulates the Transformer system correctly. Consequently, the above Turing machine correctly simulates the behavior of (T, D, C) . For the space bound, the length of the work-tape portion of $r^{(t)}$ is at most N , and the simulation additionally requires constant space to simulate one Transformer decoding step. Hence, the total space usage is constant. $\square$

A standard fact of regular languages is that $\text{REG} = \text{DSPACE}(1) = \text{NSPACE}(1)$ (see, e.g., (Gasarch, 2015; Rothvoss, 2024)), where REG denotes the class of regular languages, and $\text{NSPACE}(1)$ denotes constant space on a nondeterministic Turing machine. Therefore, when C is summarization-style and the Transformer’s context-window length, depth, and numerical precision are fixed constants (i.e., do not scale with the input length n), the system (T, D, C) can recognize only regular languages (assuming it halts), even if decoding is non-deterministic. In particular, such a fixed system cannot recognize non-regular languages such as equality $\{x\#x : x \in \Sigma^*\}$, palindromes $\{x\#x^R : x \in \Sigma^*\}$, or binary addition $\{\text{bin}(x)\#\text{bin}(y)\#\text{bin}(z) : x + y = z\}$.

Another point worth noting is that in our construction, the workspace uses space s that is at least of the same order as N (with an additional contribution coming from the space needed to simulate the Transformer). When translating this into an equivalent finite-state automaton, the resulting upper bound on the number of states grows exponentially in s , which in a sense provides an alternative perspective on scaling laws (Kaplan et al., 2020). Although this does not change the theoretical fact that the decidable languages remain regular, the languages the system can decide become more complex from the perspective of the number of states required by the corresponding finite-state automaton.

Appending-Style Context Management. Next, we discuss the computational power of appending-style context management.

Proposition 5.2. *Any fixed Transformer system (T, D, C) , where C is an appending-style context manager, can be simulated by a deterministic Turing machine using space linear in the input length n .*

Proof. The construction is straightforward. At initialization, the Turing machine copies the entire current sequence to the work tape. At decoding step t , it copies the first N tokens

into a workspace region starting from cell $n + 1$, simulates one Transformer decoding step, and obtains $\hat{x}_{t+1}$. It writes $\hat{x}_{t+1}$ into cell $n + 1$, shifts the content in cells 2 through $n + 1$ one position to the left, and clears all other cells on the work tape. $\square$

The proposition above shows that any fixed Transformer system with appending-style context management can be simulated by a linear-space Turing machine. We can also show the converse direction. Before stating the proposition and its proof, we briefly review the extended autoregressive decoding system defined by Schuurmans et al. (Schuurmans et al., 2024). Their N -gram extended autoregressive decoding is based on a sliding window. Initially, the current token string is set to the input $x = x_1 \cdots x_n$. At each decoding step, the first N tokens are fed into a fixed decoding function $M : \Sigma^N \rightarrow \Sigma^*$, which outputs a decoded token string (and the process halts if the output falls in a designated halting set). The decoded string is appended to the end of the current sequence, and the first token is removed. When the output length of M is at most K , the resulting system is called an (N, K) -restricted system. We also need the following lemma (Lemma 5.3), whose proof is deferred to Appendix B.

Lemma 5.3. *Let Σ be a finite alphabet. For any function $f : \Sigma^2 \rightarrow \Sigma$, there exists a decoder-only Transformer T such that for every length-2 context $(a, b) \in \Sigma^2$, greedy decoding outputs $f(a, b)$.*

We are now ready to state and prove the converse direction.

Proposition 5.4. *Any deterministic Turing machine that uses linear space can be simulated by a fixed Transformer system (T, D, C) , where T has context-window length 2 and C is an appending-style context manager.*

Proof. By the definition of an (N, K) -restricted system, our fixed Transformer system with an appending-style context manager is a special case with $K = 1$, where the decoding function M is implemented by the composition of a Transformer T and a decoding rule D^4 . Schuurmans et al. (Schuurmans et al., 2024) show that the computation of any linear-space Turing machine can be simulated by a $(2, 1)$ -restricted system. According to Lemma 5.3, any function $f : \Sigma^2 \rightarrow \Sigma$ can be realized by a Transformer T with context-window length 2 coupled with the greedy decoding rule, and the claim follows. $\square$

Propositions 5.2 and 5.4 suggest that a single fixed Transformer system with appending-style context management

⁴In the definition of (Schuurmans et al., 2024), when $K = 1$ the decoding output of M can contain zero or one token. In our appending-style context management, this can be modeled by including an empty symbol in Σ and ignoring it when appending.

has the same computational power as a linear-space Turing machine. In particular, it can decide exactly the class of deterministic context-sensitive languages, $\text{DCSL} = \text{DSPACE}(n)$ (see, e.g., (Ibarra, 1991; Otto, 2006)). Schuurmans et al. (Schuurmans et al., 2024) also show that a $(2, 2)$ -restricted system is Turing-complete. If one could extend Lemma 5.3 to show that a context-window-2 Transformer can realize any function $\Sigma^2 \rightarrow \Sigma^2$ by applying greedy decoding to the two output positions $Y(1, :)$ and $Y(2, :)$, then this would yield a Turing-complete system as well. Note that this changes the decoding interface from next-token decoding to next-two-tokens decoding. In Appendix C, we briefly discuss how one might preserve next-token decoding by modifying the context manager to perform two decoding steps and then discard one of the produced tokens.

Table 2 summarizes the computational power of a fixed Transformer system under different context managers. We conclude that different context managers can endow the same underlying model with markedly different computational power, which supports our position: **the Turing-completeness of real-world autoregressive Transformers relies heavily on context management.**

Table 2. Computational power under different context managers. Here $\equiv$ denotes equivalence and $\leq$ denotes no more powerful than.

Work	Context Management	Power
(Schuurmans, 2023)	read / write memory	$\equiv \text{TM}$
(Schuurmans et al., 2024)	$(2, 2)$ -restricted sys.	$\equiv \text{TM}$
(Schuurmans et al., 2024)	$(2, 1)$ -restricted sys.	$\equiv O(n)$ -space TM
Ours	appending-style	$\equiv O(n)$ -space TM
Ours	summarization-style	$\leq O(1)$ -space TM

6. Summary and Call to Action

In this paper, we formalized the computation of a fixed Transformer system and distinguished two settings for the Turing-completeness of autoregressive Transformers. We showed that the scaling-family setting is theoretically misaligned with the notion of Turing-completeness and, in practice, does not match how real-world LLMs are deployed and used. We then analyzed the fixed-system setting and demonstrated that context management can substantially affect the computational power of Transformers.

We conclude with three calls to action.

- First, claims about Transformer Turing-completeness should **explicitly state their computational setting and**

assumptions. Scaling-family results are valuable for understanding resource requirements, such as context length, precision, and depth, but should not be interpreted as Turing-completeness of a fixed real-world Transformer system.

- Second, since real-world deployed LLM systems consist of a fixed-context-window, fixed-precision Transformer together with a particular context-management method, theoretical work should **devote more attention to the capabilities of the overall system obtained by combining different context-management methods with a fixed Transformer.**
- Third, theoretical works should **look beyond qualitative Turing-completeness analyses under idealized assumptions, and complement them with capability claims stated in terms of explicit resource budgets and learnability criteria**, since Turing-completeness only concerns whether a function is computable under a specified encoding, and does not by itself answer whether the model can acquire, generalize, or robustly use the relevant solution.

Acknowledgments

This research was supported in part by National Natural Science Foundation of China (No. L2524018, No. U2241212, No. 92470128, No. 62472430) and by Industrial AI Solutions, Li Auto Inc. We also wish to acknowledge the support provided by the fund for building world-class universities (disciplines) of Renmin University of China, by Engineering Research Center of Next-Generation Intelligent Search and Recommendation, Ministry of Education, by Intelligent Social Governance Interdisciplinary Platform, Major Innovation & Planning Interdisciplinary Platform for the “Double-First Class” Initiative, Public Policy and Decision-making Research Lab, and Public Computing Cloud, Renmin University of China.

The work was partially done at Gaoling School of Artificial Intelligence, Beijing Key Laboratory of Research on Large Models and Intelligent Governance, Engineering Research Center of Next-Generation Intelligent Search and Recommendation, MOE, and Pazhou Laboratory (Huangpu), Guangzhou, Guangdong 510555, China.

We would also like to thank Hessameddin Akhlaghpour for the blog post *Are Transformers Turing-Complete? A Good Disguise Is All You Need*, which inspired us to further examine and carefully distinguish the assumptions in existing theoretical works that align with real-world practice from those that do not. It also motivated us to investigate, under assumptions that better reflect real-world Transformers, how the capabilities of these models are closely tied to context management. We also thank the anonymous reviewers for their valuable comments.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Akhlaghpour, H. Are transformers turing-complete? a good disguise is all you need., October 2024. URL <https://lifeiscomputation.com/transformers-are-not-turing-complete/>. Blog post. Updated: 2024-10-14.
- Anthropic. Commands. Claude Code Documentation, 2026a. URL <https://code.claude.com/docs/en/commands>. Accessed: 2026-05-12.
- Anthropic. Tool use with claude. Claude Docs, 2026b. URL <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>. Accessed: 2026-05-12.
- Back De Luca, A. and Fountoulakis, K. Simulation of graph algorithms with looped transformers. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 2319–2363, 2024.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Bhattachamishra, S., Patel, A., and Goyal, N. On the computational power of transformers and its implications in sequence modeling. In *Proceedings of the 24th Conference on Computational Natural Language Learning*, pp. 455–475, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chevalier, A., Wettig, A., Ajith, A., and Chen, D. Adapting language models to compress contexts. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 3829–3846, Singapore, 12 2023. Association for Computational Linguistics.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, L. Universal transformers. In *International Conference on Learning Representations*, 2019.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- Gasarch, W. Classifying problems into complexity classes. University of Maryland, College Park, 11 2015. URL <https://www.cs.umd.edu/~gasarch/papers/classcomp.pdf>. Accessed: 2026-01-22.
- Ge, T., Jing, H., Wang, L., Wang, X., Chen, S.-Q., and Wei, F. In-context autoencoder for context compression in a large language model. In *The Twelfth International Conference on Learning Representations*, 2024.
- Gemini Team, Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Gemini Team, Georgiev, P., Lei, V. I., Burnell, R., Bai, L., Gulati, A., Tanzer, G., Vincent, D., Pan, Z., Wang, S., et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. Looped transformers as programmable computers. In *International Conference on Machine Learning*, pp. 11398–11442. PMLR, 2023.
- Google. CLI Commands. Gemini CLI Documentation, 2026a. URL <https://google-gemini.github.io/gemini-cli/docs/cli/commands.html>. Accessed: 2026-05-12.
- Google. Function calling with the Gemini API. Gemini API Docs, 2026b. URL <https://ai.google.dev/gemini-api/docs/function-calling>. Accessed: 2026-05-12.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. DeepSeek-R1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Guu, K., Lee, K., Tung, Z., Pasupat, P., and Chang, M. Retrieval augmented language model pre-training. In *International conference on machine learning*, pp. 3929–3938. PMLR, 2020.
- Hou, K., Brandfonbrener, D., Kakade, S. M., Jelassi, S., and Malach, E. Universal length generalization with turing programs. In *International Conference on Machine Learning*, pp. 23873–23893. PMLR, 2025.

- Ibarra, O. H. On resetting dlba's. *ACM SIGACT News*, 22 (1):62–63, 1991.
- Jiang, H., Hahn, M., Zetzsche, G., and Lin, A. W. Soft-max transformers are turing-complete. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., and Zettlemoyer, L. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pp. 7871–7880, 2020a.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020b.
- Li, Q. and Wang, Y. Constant bit-size transformers are turing complete. *Advances in Neural Information Processing Systems*, 38:62273–62292, 2025.
- Li, Z., Liu, H., Zhou, D., and Ma, T. Chain of thought empowers transformers to solve inherently serial problems. In *The Twelfth International Conference on Learning Representations*, 2024.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Malach, E. Auto-regressive next-token predictors are universal learners. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 34417–34431, 2024.
- Merrill, W. and Sabharwal, A. The expressive power of transformers with chain of thought. In *The Twelfth International Conference on Learning Representations*, 2024.
- Nowak, F., Svete, A., Butoi, A., and Cotterell, R. On the representational capacity of neural language models with chain-of-thought reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12510–12548, 2024.
- OpenAI. Slash commands in Codex CLI. OpenAI Developers Documentation, 2026a.
URL <https://developers.openai.com/codex/cli/slash-commands#keep-transcripts-lean-with-compact>. Accessed: 2026-05-12.
- OpenAI. Function calling. OpenAI API Docs, 2026b. URL <https://developers.openai.com/api/docs/guides/function-calling>. Accessed: 2026-05-12.
- Otto, F. Restarting automata. In *Recent Advances in Formal Languages and Applications*, volume 25, pp. 269–303. Springer, 2006.
- Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. MemGPT: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Pérez, J., Marinković, J., and Barceló, P. On the turing completeness of modern neural network architectures. In *International Conference on Learning Representations*, 2019.
- Qiu, R., Xu, Z., Bao, W., and Tong, H. Ask, and it shall be given: On the turing completeness of prompting. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.
- Roberts, J. How powerful are decoder-only transformer neural models? In *2024 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. IEEE, 2024.
- Rothvoss, T. Computational complexity i. CSE 531 (Winter 2024) lecture notes, University of Washington, 2024. URL <https://sites.math.washington.edu/~rothvoss/archive/lecturenotes/complexity-CSE531-Winter2024.pdf>. Accessed: 2026-05-12.
- Savage, J. E. Computational work and time on finite machines. *Journal of the ACM (JACM)*, 19(4):660–674, 1972.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- Schuurmans, D. Memory augmented large language models are computationally universal. *arXiv preprint arXiv:2301.04589*, 2023.

- Schuermans, D., Dai, H., and Zanini, F. Autoregressive large language models are computationally universal. *arXiv preprint arXiv:2410.03170*, 2024.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Turing, A. M. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1937.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Appendices

A. More Details of Transformers

We first introduce our notation for vectors and matrices. Bold lowercase letters such as $\mathbf{x}$ denote vectors (column vectors unless stated otherwise), and bold uppercase letters such as $\mathbf{X}$ denote matrices. We write $\mathbf{x}(i)$ for the i -th entry of $\mathbf{x}$. For a matrix $\mathbf{X}$, $\mathbf{X}(i, :)$ denotes the i -th row (as a row vector), $\mathbf{X}(:, j)$ denotes the j -th column (as a column vector), and $\mathbf{X}(i, j)$ denotes the (i, j) -th entry.

Below, we describe in detail a single forward pass of a Transformer:

- **Embedding:** A Transformer first maps each token ID to a token embedding vector of dimension d . This is done by an embedding layer that applies a pointwise lookup-table function $g : [|\Sigma|] \rightarrow \mathbb{R}^d$ to each token ID. In some architectures, positional information is incorporated at the embedding stage by adding a positional encoding (PE) to the embedding vectors, such as the sinusoidal PE in the vanilla Transformer (Vaswani et al., 2017) or the learned PE in BERT (Devlin et al., 2019), to make the model aware of token positions. In other cases, positional information is introduced within self-attention (described later), for example via RoPE (Su et al., 2024), which is used by many LLMs. In summary, the embedding layer can be formulated as a mapping $\text{emb} : [|\Sigma|]^N \times \mathbb{N}^N \rightarrow \mathbb{R}^{N \times d}$, which maps at most N token IDs (where N is the context window length, i.e., the maximum input length accepted by the Transformer) together with their position indices to an embedding matrix.
- **Multi-head masked self-attention decoding:** The self-attention mechanism is one of the most fundamental components of Transformers. First, in the ℓ -th self-attention decoding layer, we apply linear maps to obtain the query, key, and value matrices $\mathbf{Q}^{(\ell)}, \mathbf{K}^{(\ell)}, \mathbf{V}^{(\ell)} \in \mathbb{R}^{N \times d}$, namely $\mathbf{Q}^{(\ell)} = \mathbf{X}^{(\ell)} \mathbf{W}_Q^{(\ell)}$, $\mathbf{K}^{(\ell)} = \mathbf{X}^{(\ell)} \mathbf{W}_K^{(\ell)}$, and $\mathbf{V}^{(\ell)} = \mathbf{X}^{(\ell)} \mathbf{W}_V^{(\ell)}$, where $\mathbf{X}^{(\ell)} \in \mathbb{R}^{N \times d}$ is the input embedding matrix and $\mathbf{W}_Q^{(\ell)}, \mathbf{W}_K^{(\ell)}, \mathbf{W}_V^{(\ell)} \in \mathbb{R}^{d \times d}$ are weight matrices. For multi-head masked self-attention, we split each matrix along the embedding dimension into H submatrices $\{\mathbf{Q}_h^{(\ell)}, \mathbf{K}_h^{(\ell)}, \mathbf{V}_h^{(\ell)} \in \mathbb{R}^{N \times d_h} : h \in [H]\}$ such that $\sum_{h \in [H]} d_h = d$, and assign them to H attention heads. On each head, the masked self-attention output is computed as

$$\tilde{\mathbf{X}}_h^{(\ell)} = \text{softmax} \left(\frac{\mathbf{Q}_h^{(\ell)} (\mathbf{K}_h^{(\ell)})^\top}{\sqrt{d_h}} + \mathbf{M} \right) \mathbf{V}_h^{(\ell)}, \quad (1)$$

where $\mathbf{M} \in \{0, -\infty\}^{N \times N}$ is the causal mask, defined by $\mathbf{M}(i, j) = -\infty$ if and only if $i < j$. We then concatenate $\{\tilde{\mathbf{X}}_h^{(\ell)} : h \in [H]\}$ to recover the original shape, add the result to the input $\mathbf{X}^{(\ell)}$ via a residual connection, and feed it into an MLP (also with a residual connection) to obtain the layer output $\mathbf{X}^{(\ell+1)}$.

- **Classification head:** After the final decoding layer, we map the output matrix $\mathbf{X}^{(L)} \in \mathbb{R}^{N \times d}$ through a linear projection $\mathbf{W} \in \mathbb{R}^{d \times |\Sigma|}$ to obtain the token logits at each position, i.e., $\mathbf{Y} = \mathbf{X}^{(L)} \mathbf{W} \in \mathbb{R}^{N \times |\Sigma|}$.

B. Proof of Lemma 5.3

Lemma B.1. *Let Σ be a finite alphabet. For any function $f : \Sigma^2 \rightarrow \Sigma$, there exists a decoder-only Transformer T such that for every length-2 context $(a, b) \in \Sigma^2$, greedy decoding outputs $f(a, b)$.*

Proof. Without loss of generality, assume $\Sigma = \{1, 2, \dots, K\}$. Let $d_{\text{model}} = K + 2K + K^2$, and partition each token embedding (row) vector $\mathbf{x} \in \mathbb{R}^{1 \times d_{\text{model}}}$ into three blocks, $\mathbf{x} := (\mathbf{x}^{\text{tok}}; \mathbf{x}^{\text{feat}}; \mathbf{x}^{\text{pair}})$, with dimensions K , $2K$, and K^2 , respectively.

We construct a one-layer decoder-only Transformer that consists of (i) multi-head self-attention with two heads (denoted prev and self), each with $d_k = K$, followed by a residual connection, and (ii) a position-wise feed-forward network (FFN),

⁵All arithmetic operations in this proof can be carried out to constant precision (with constants depending only on $|\Sigma|$); we use $\mathbb{R}$ only for notational convenience.

followed by a residual connection. We do not use positional encodings, LayerNorm, or dropout. The construction is summarized as follows:

- **Embedding.** For a token a , set its embedding to $(e_a; \mathbf{0}; \mathbf{0})$, where $e_a \in \mathbb{R}^{1 \times K}$ is the one-hot vector corresponding to a .
- **Self-attention.** After applying self-attention to the two input tokens, the representation at position 2 has the form $(*; (\mathbf{o}_2^{(\text{prev})}, \mathbf{o}_2^{(\text{self})}); \mathbf{0})$.
- **FFN.** Using the affine map $\mathbf{X}\mathbf{W}_1 + \mathbf{1}\mathbf{b}_1^\top$, we first recover $(*; (e_a, e_b); \mathbf{0})$ from $(*; (\mathbf{o}^{(\text{prev})}, \mathbf{o}^{(\text{self})}); \mathbf{0})$, and then apply a second affine map to produce $(*; *; e_{(a-1)K+b})$.
- **Output head.** Finally, we map $e_{(a-1)K+b}$ in the pair block to logits using the classification head.

The details are as follows:

Step 1: Embedding. Given input tokens $(a, b) \in \Sigma^2$, define the two token representations

$$\mathbf{X} := \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} e_a & \mathbf{0} & \mathbf{0} \\ e_b & \mathbf{0} & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{2 \times d_{\text{model}}},$$

where $\mathbf{x}_1 = (e_a; \mathbf{0}; \mathbf{0})$ and $\mathbf{x}_2 = (e_b; \mathbf{0}; \mathbf{0})$.

Step 2: Two-Head Self-Attention. Let the query matrices be

$$\mathbf{W}_Q^{(\text{prev})} := \begin{bmatrix} \sqrt{K}(\mathbf{J} - \mathbf{I}_K) \\ \mathbf{0}_{2K \times K} \\ \mathbf{0}_{K^2 \times K} \end{bmatrix}, \quad \mathbf{W}_Q^{(\text{self})} := \begin{bmatrix} \sqrt{K}\mathbf{I}_K \\ \mathbf{0}_{2K \times K} \\ \mathbf{0}_{K^2 \times K} \end{bmatrix},$$

and for each head $h \in \{\text{prev}, \text{self}\}$ define

$$\mathbf{W}_K^{(h)} = \mathbf{W}_V^{(h)} := \begin{bmatrix} \mathbf{I}_K \\ \mathbf{0}_{2K \times K} \\ \mathbf{0}_{K^2 \times K} \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times K}.$$

For head h , the query at position 2 is

$$\mathbf{q}_2^{(h)} := \mathbf{x}_2 \mathbf{W}_Q^{(h)} = \begin{cases} \sqrt{K}(\mathbf{1} - e_b), & h = \text{prev}, \\ \sqrt{K}e_b, & h = \text{self}, \end{cases}$$

and for $t \in \{1, 2\}$ the keys and values satisfy

$$\mathbf{k}_t^{(h)} := \mathbf{x}_t \mathbf{W}_K^{(h)} = \mathbf{v}_t^{(h)} := \mathbf{x}_t \mathbf{W}_V^{(h)} = \begin{cases} e_a, & t = 1, \\ e_b, & t = 2. \end{cases}$$

Define the attention weights

$$\begin{aligned} \alpha_{2,t}^{(h)} &:= \frac{\exp(\langle \mathbf{q}_2^{(h)}, \mathbf{k}_t^{(h)} \rangle / \sqrt{K})}{\sum_{u \in \{1,2\}} \exp(\langle \mathbf{q}_2^{(h)}, \mathbf{k}_u^{(h)} \rangle / \sqrt{K})} = \frac{\exp(\langle \mathbf{q}_2^{(h)}, \mathbf{k}_t^{(h)} \rangle / \sqrt{K})}{\exp(\langle \mathbf{q}_2^{(h)}, e_a \rangle / \sqrt{K}) + \exp(\langle \mathbf{q}_2^{(h)}, e_b \rangle / \sqrt{K})} \\ &= \begin{cases} \frac{\exp(\langle \mathbf{1} - e_b, \mathbf{k}_t^{(h)} \rangle)}{\exp(\mathbb{I}[a \neq b]) + 1}, & h = \text{prev}, \\ \frac{\exp(\langle e_b, \mathbf{k}_t^{(h)} \rangle)}{\exp(\mathbb{I}[a = b]) + e}, & h = \text{self}. \end{cases} \end{aligned}$$

We next compute the head outputs at position 2, denoted $\mathbf{o}_2^{(h)} \in \mathbb{R}^{1 \times K}$. Since

$$\mathbf{o}_2^{(h)} := \sum_{t \in \{1,2\}} \alpha_{2,t}^{(h)} \mathbf{v}_t^{(h)} = \alpha_{2,1}^{(h)} \mathbf{e}_a + \alpha_{2,2}^{(h)} \mathbf{e}_b,$$

we obtain

$$\mathbf{o}_2^{(\text{prev})} = \begin{cases} \frac{e}{e+1} \mathbf{e}_a + \frac{1}{e+1} \mathbf{e}_b, & a \neq b, \\ \frac{1}{2} \mathbf{e}_a + \frac{1}{2} \mathbf{e}_b, & a = b, \end{cases} \quad \mathbf{o}_2^{(\text{self})} = \begin{cases} \frac{1}{e+1} \mathbf{e}_a + \frac{e}{e+1} \mathbf{e}_b, & a \neq b, \\ \frac{1}{2} \mathbf{e}_a + \frac{1}{2} \mathbf{e}_b, & a = b. \end{cases}$$

Let the concatenated head outputs be

$$\text{Attn}(\mathbf{X}) := \begin{bmatrix} \mathbf{o}_1^{(\text{prev})} & \mathbf{o}_1^{(\text{self})} \\ \mathbf{o}_2^{(\text{prev})} & \mathbf{o}_2^{(\text{self})} \end{bmatrix} \in \mathbb{R}^{2 \times 2K}.$$

Choose the attention output projection $\mathbf{W}_O := [\mathbf{0}_{2K \times K} \quad \mathbf{I}_{2K} \quad \mathbf{0}_{2K \times K^2}] \in \mathbb{R}^{2K \times d_{\text{model}}}$ so that $\text{Attn}(\mathbf{X})\mathbf{W}_O$ writes the two K -dimensional head outputs into the middle feature block (and zeros elsewhere). Let $\mathbf{Z} := \mathbf{X} + \text{Attn}(\mathbf{X})\mathbf{W}_O$ be the post-attention representation matrix. Then

$$\mathbf{Z}(2, :) = \left(\mathbf{e}_b; \left(\mathbf{o}_2^{(\text{prev})}; \mathbf{o}_2^{(\text{self})} \right); \mathbf{0} \right).$$

Step 3: The FFN Builds a Pair Indicator. A key observation is that $\mathbf{e}_a = \frac{e}{e-1} \mathbf{o}_2^{(\text{prev})} - \frac{1}{e-1} \mathbf{o}_2^{(\text{self})}$ and $\mathbf{e}_b = \frac{e}{e-1} \mathbf{o}_2^{(\text{self})} - \frac{1}{e-1} \mathbf{o}_2^{(\text{prev})}$. These identities follow by inverting the corresponding linear system when $a \neq b$, and one can verify that they also hold for $a = b$.

Let

$$\mathbf{W}_A := \begin{bmatrix} \mathbf{0}_{K \times K} & \mathbf{0}_{K \times K} & \mathbf{0}_{K \times K} & \mathbf{0}_{K \times K^2} \\ \mathbf{0}_{K \times K} & \frac{e}{e-1} \mathbf{I}_{K \times K} & -\frac{1}{e-1} \mathbf{I}_{K \times K} & \mathbf{0}_{K \times K^2} \\ \mathbf{0}_{K \times K} & -\frac{1}{e-1} \mathbf{I}_{K \times K} & \frac{e}{e-1} \mathbf{I}_{K \times K} & \mathbf{0}_{K \times K^2} \\ \mathbf{0}_{K^2 \times K} & \mathbf{0}_{K^2 \times K} & \mathbf{0}_{K^2 \times K} & \mathbf{0}_{K^2 \times K^2} \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}},$$

so that $(\mathbf{Z}\mathbf{W}_A)(2, :) = (\mathbf{0}; \mathbf{e}_a; \mathbf{e}_b; \mathbf{0})$.

Next, let

$$\mathbf{W}_B := \begin{bmatrix} \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{e}_1^\top & \mathbf{e}_1^\top & \cdots & \mathbf{e}_1^\top & \mathbf{e}_2^\top & \cdots & \mathbf{e}_K^\top & \cdots & \mathbf{e}_K^\top \\ \mathbf{e}_1^\top & \mathbf{e}_2^\top & \cdots & \mathbf{e}_K^\top & \mathbf{e}_1^\top & \cdots & \mathbf{e}_1^\top & \cdots & \mathbf{e}_K^\top \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \cdots & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times K^2}.$$

Equivalently, the $((i-1)K+j)$ -th column of $\mathbf{W}_B$ is $(\mathbf{0}; \mathbf{e}_i; \mathbf{e}_j; \mathbf{0})^\top \in \mathbb{R}^{d_{\text{model}}}$, and hence for any row vector $\mathbf{x} \in \mathbb{R}^{1 \times d_{\text{model}}}$, $(\mathbf{x}\mathbf{W}_B)((i-1)K+j) = \mathbf{x}(K+i) + \mathbf{x}(2K+j)$.

Let $\mathbf{W}_1 := \mathbf{W}_A\mathbf{W}_B$ and $\mathbf{b}_1 := -\mathbf{1}$. Then $(\text{ReLU}(\mathbf{Z}\mathbf{W}_1 + \mathbf{1}\mathbf{b}_1^\top))(2, :) = \mathbf{e}_{(a-1)K+b} \in \mathbb{R}^{1 \times K^2}$.

Finally, choose the second-layer parameters as $\mathbf{W}_2 := [\mathbf{0}_{K^2 \times K} \quad \mathbf{0}_{K^2 \times 2K} \quad \mathbf{I}_{K^2}] \in \mathbb{R}^{K^2 \times d_{\text{model}}}$ and $\mathbf{b}_2 := \mathbf{0}$. Let

$$\mathbf{X}^{(1)} := \mathbf{Z} + \left(\text{ReLU}(\mathbf{Z}\mathbf{W}_1 + \mathbf{1}\mathbf{b}_1^\top) \mathbf{W}_2 + \mathbf{1}\mathbf{b}_2^\top \right)$$

denote the FFN output with the residual connection. Then

$$\mathbf{X}^{(1)}(2, :) = \left(\mathbf{e}_b; \left(\mathbf{o}_2^{(\text{prev})}; \mathbf{o}_2^{(\text{self})} \right); \mathbf{e}_{(a-1)K+b} \right).$$

Step 4: Output. Define $\mathbf{W} \in \mathbb{R}^{d_{\text{model}} \times K}$ such that $\mathbf{W}(i, j) = 0$ for $i \in \{1, 2, \dots, 3K\}$ and $j \in \{1, 2, \dots, K\}$, and $\mathbf{W}(3K+(i-1)K+j, k) = \mathbb{I}[f(i, j) = k]$ for $i, j, k \in \{1, 2, \dots, K\}$. Let $\mathbf{Y} := \mathbf{X}^{(1)}\mathbf{W}$. Then for any $k \in \{1, 2, \dots, K\}$,

$$\mathbf{Y}(2, k) = \sum_{\substack{(i,j) \in \{1, \dots, K\}^2: \\ f(i,j)=k}} \mathbf{X}^{(1)}(2, 3K+(i-1)K+j) = \mathbf{e}_{f(a,b)}(k).$$

Therefore, the greedy decoding output $\arg \max \mathbf{Y}(2, :)$ is unique and equals $f(a, b)$. $\square$

C. Discussion on Modifying the Context Manager to Simulate Next-Two-Tokens Decoding

In this appendix, we show how to simulate a $(2, 2)$ -restricted system while preserving the interface in which each Transformer call decodes a single token. Let Σ be a finite alphabet, and let $\langle 1 \rangle, \langle 2 \rangle \notin \Sigma$ be two control tokens. Define the extended alphabet $\bar{\Sigma} := \Sigma \cup \{\langle 1 \rangle, \langle 2 \rangle\}$. Assume that for every function $f : \Sigma^2 \rightarrow \Sigma^2$, writing $f(a, b) = (f_1(a, b), f_2(a, b))$, there exists a context-window-3 Transformer over $\bar{\Sigma}$ such that for all $(a, b) \in \Sigma^2$, greedy decoding on input $(\langle 1 \rangle, a, b)$ outputs $f_1(a, b)$, and greedy decoding on input $(\langle 2 \rangle, a, b)$ outputs $f_2(a, b)$. Under this assumption, we can simulate any $(2, 2)$ -restricted system.

Concretely, consider a $(2, 2)$ -restricted system specified by a function $M : \Sigma^2 \rightarrow \Sigma^2$.⁶ Applying the assumption with $f \equiv M$, we obtain a Transformer T that produces the two components of $M(a, b)$ via the control tokens $\langle 1 \rangle$ and $\langle 2 \rangle$. Now consider step t of the simulated $(2, 2)$ -restricted system, where the current window content is (a, b) . The modified context manager first feeds $(\langle 1 \rangle, a, b)$ to T and appends the decoded token to the end of the current sequence. It then feeds $(\langle 2 \rangle, a, b)$ to T and appends the decoded token as well. Finally, it deletes the first token of the current sequence, thereby shifting the window forward by one position. This procedure exactly reproduces one step of the $(2, 2)$ -restricted system using two single-token decoding calls. Therefore, it suffices to establish that the assumption above holds. We note that this follows by a direct extension of Lemma 5.3 to the context-window-3 setting, and we omit the details.

⁶Schuurmans et al. (Schuurmans et al., 2024) require the output length of M to be at most 2. If M outputs the empty string, we treat it as (ϵ, ϵ) . If M outputs a single token a , we treat it as (a, ϵ) , where ϵ is a special empty symbol that is ignored when appending.